

AFRILANG Stdlib

std/c/graphmst

Bibliothèque AFRILANG `std/c/graphmst` (niveau complex, catégorie complex). Elle expose 6 fonction(s) :
mstWeightPrim, m

```
`import "std/c/graphmst" · `use graphmst`
```

- `mstWeightPrim(adj list number, n number) ? number`
- `mstWeightKruskal(adj list number, n number) ? number`
- `mstEdges(adj list number, n number) ? number`
- `compareMstAlgos(adj list number, n number) ? number`
- `forestMstWeight(adj list number, n number) ? number`
- `isConnectedForMst(adj list number, n number) ? bool`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG ``std/c/graphmst`` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : `mstWeightPrim`, `m`

```
`import "std/c/graphmst"` · `use graphmst`  
- `mstWeightPrim(adj list number, n number) ? number`  
- `mstWeightKruskal(adj list number, n number) ? number`  
- `mstEdges(adj list number, n number) ? number`  
- `compareMstAlgos(adj list number, n number) ? number`  
- `forestMstWeight(adj list number, n number) ? number`  
- `isConnectedForMst(adj list number, n number) ? bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/graphmst"  
use graphmst
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? mstWeightPrim(adj list number, n number) ? number  
? mstWeightKruskal(adj list number, n number) ? number  
? mstEdges(adj list number, n number) ? number  
? compareMstAlgos(adj list number, n number) ? number  
? forestMstWeight(adj list number, n number) ? number  
? isConnectedForMst(adj list number, n number) ? bool
```

4. Exemples d'utilisation

```
import "std/c/graphmst"  
use graphmst  
  
create result = mstWeightPrim(/* args */)   
say result  
  
create result = mstWeightKruskal(/* args */)   
say result  
  
create result = mstEdges(/* args */)   
say result  
  
create result = compareMstAlgos(/* args */)   
say result  
  
create result = forestMstWeight(/* args */)   
say result  
  
create result = isConnectedForMst(/* args */)   
say result
```

5. Bonnes pratiques

? Préférez ``import "std/c/graphmst"`` en tête de fichier.
? Lancez ``afrilang check`` avant de livrer.
? Combinez avec les modules `stdlib` de la même catégorie.
? Voir `/stdlib/` pour le source live et les démos playground.
? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module graphmst  
  
function mstWeightPrim(adj list number, n number) returns number  
end
```

```
function mstWeightKruskal(adj list number, n number) returns number
end

function mstEdges(adj list number, n number) returns number
end

function compareMstAlgos(adj list number, n number) returns number
end

function forestMstWeight(adj list number, n number) returns number
end

function isConnectedForMst(adj list number, n number) returns bool
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/graphmst` (tier complex, category complex). Exports 6 function(s): mstWeightPrim, mstWeightKrusk

```
`import "std/c/graphmst"` . `use graphmst`  
- `mstWeightPrim(adj list number, n number) ? number`  
- `mstWeightKruskal(adj list number, n number) ? number`  
- `mstEdges(adj list number, n number) ? number`  
- `compareMstAlgos(adj list number, n number) ? number`  
- `forestMstWeight(adj list number, n number) ? number`  
- `isConnectedForMst(adj list number, n number) ? bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/graphmst"  
use graphmst
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? mstWeightPrim(adj list number, n number) ? number  
? mstWeightKruskal(adj list number, n number) ? number  
? mstEdges(adj list number, n number) ? number  
? compareMstAlgos(adj list number, n number) ? number  
? forestMstWeight(adj list number, n number) ? number  
? isConnectedForMst(adj list number, n number) ? bool
```

4. Usage examples

```
import "std/c/graphmst"  
use graphmst  
  
create result = mstWeightPrim(/* args */)   
say result  
  
create result = mstWeightKruskal(/* args */)   
say result  
  
create result = mstEdges(/* args */)   
say result  
  
create result = compareMstAlgos(/* args */)   
say result  
  
create result = forestMstWeight(/* args */)   
say result  
  
create result = isConnectedForMst(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/graphmst"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module graphmst  
  
function mstWeightPrim(adj list number, n number) returns number  
end
```

```
function mstWeightKruskal(adj list number, n number) returns number
end

function mstEdges(adj list number, n number) returns number
end

function compareMstAlgos(adj list number, n number) returns number
end

function forestMstWeight(adj list number, n number) returns number
end

function isConnectedForMst(adj list number, n number) returns bool
end
end
```